

Clasificación de variedades de arroz mediante regresión logística

Marco Iván Flores Villanueva
Instituto Tecnológico y de Estudios Superiores de Monterrey
Querétaro, México
A01276586@tec.mx

Abstract — Este documento presenta la implementación manual de un modelo de regresión logística para clasificar granos de arroz de las variedades Cammeo y Osmancik utilizando sus características físicas. El dataset a procesar contiene 3810 observaciones y siete variables numéricas obtenidas a partir de imágenes procesadas de granos de arroz. Se realizó un EDA para evaluar la calidad de la información, la distribución de las clases y las relaciones entre las variables, y posteriormente, se separó la variable objetivo, se dividieron los datos en conjuntos de entrenamiento y prueba, y se aplicó escalamiento para mejorar la estabilidad del gradiente descendente. El modelo fue implementado sin el uso de frameworks de Machine Learning, utilizando la función *sigmoid*, *cross-entropy* y gradiente descendente para optimizar los parámetros. Con un learning rate de 0.03 en 3000 epochs, el modelo logró una precisión del 88.16% en entrenamiento y 87.66% en prueba.

Keywords — *rice, logistic regression, sigmoid, cross-entropy, gradient descent, Turkey*

I. INTRODUCCIÓN

Los algoritmos de machine learning permiten construir modelos capaces de aprender relaciones entre variables de entrada y una variable objetivo a partir de ejemplos que los entrenan. En ese ámbito, los problemas de clasificación buscan asignar cada observación a una categoría del dataset, y cuando sólo hay 2 clases, la regresión logística es una de las técnicas más utilizadas por su simplicidad y facilidad de comprenderla.

Para este trabajo, decidí utilizar el dataset Rice - Cammeo and Osmancik, compuesto por 3810 granos de arroz clasificados en 2 variedades. A partir de las imágenes procesadas de los granos, se identificaron siete características físicas: área, perímetro, longitud del eje mayor, longitud del eje menor, excentricidad, área convexa y extensión. Estas características permiten analizar diferencias de tamaño y forma entre ambas variedades, por lo cual resulta interesante averiguar si también son suficientes para categorizarlas completamente y poder predecir qué variedad de arroz corresponde a cada grano.

Es por esto que el objetivo de este trabajo es implementar un modelo de regresión logística, por el momento, sin utilizar frameworks de machine learning. Para eso, primero se llevó a cabo un análisis exploratorio (EDA) y una preparación de los datos, seguido de la implementación de la función *sigmoid*, la *cross-entropy* y el gradiente descendente para estimar los parámetros del modelo. Finalmente, se evaluó el desempeño con datos de entrenamiento y prueba, así como predicciones individuales sobre observaciones no utilizadas durante el entrenamiento.

II. DATASET Y EDA

A. Descripción del dataset

El dataset Rice (Cammeo and Osmancik), publicado en el UCI Machine Learning Repository en 2019, contiene 3810 observaciones correspondientes a granos de arroz de las variedades Cammeo y Osmancik. Sus datos fueron obtenidos mediante el procesamiento de imágenes de los granos, y se extrajeron siete características: *Area*, *Perimeter*, *Major Axis Length*, *Minor Axis Length*, *Eccentricity*, *Convex Area* y *Extent*. Mientras que estas variables describen las propiedades físicas de cada grano, la variable objetivo *Class* clasifica a cada observación como Cammeo u Osmancik, y es debido a eso que este dataset representa un problema de clasificación binaria que puede ser atacado con la implementación de un modelo de regresión logística.

Variable Name	Role	Type	Description	Units
Area	Feature	Integer	Returns the number of pixels within the boundaries of the rice grain	px
Perimeter	Feature	Continuous	Calculates the circumference by calculating the distance between pixels around the boundaries of the rice grain	px
Major_Axis_Length	Feature	Continuous	The longest line that can be drawn on the rice grain, i.e. the main axis distance, gives	
Minor_Axis_Length	Feature	Continuous	The shortest line that can be drawn on the rice grain, i.e. the small axis distance, gives	
Eccentricity	Feature	Continuous	It measures how round the ellipse, which has the same moments as the rice grain, is	
Convex_Area	Feature	Integer	Returns the pixel count of the smallest convex shell of the region formed by the rice grain	
Extent	Feature	Continuous	Returns the ratio of the region formed by the rice grain to the bounding box	
Class	Target	Binary	Cammeo and Osmancik	

B. Revisión de los datos y limpieza

Para tener un mejor panorama de los datos antes de construir cualquier modelo, se aplicó una revisión inicial de su estructura y calidad. El dataset importado era de 3810 observaciones y 8 columnas correspondientes a las 7 características físicas y a la clase. Fue ahí que se comprobó que las variables eran efectivamente numéricas, y que *Class* era un string.

```
RangeIndex: 3810 entries, 0 to 3809
Data columns (total 8 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   Area                3810 non-null   int64
1   Perimeter           3810 non-null   float64
2   Major Axis Length   3810 non-null   float64
3   Minor Axis Length   3810 non-null   float64
4   Eccentricity         3810 non-null   float64
5   Convex Area         3810 non-null   int64
6   Extent              3810 non-null   float64
7   Class               3810 non-null   str
```

Después se revisó si existían valores faltantes o registros duplicados, y por fortuna no era el caso, por lo que no fue necesario realizar ninguna imputación o eliminación de observaciones.

```
Valores faltantes:
Area          0
Perimeter     0
Major Axis Length  0
Minor Axis Length  0
Eccentricity   0
Convex Area    0
Extent         0
Class          0
dtype: int64

Filas duplicadas:
0
```

C. Distribución de clases

La distribución de Class fue de 2180 observaciones de Osmancik y 1630 de Cammeo, aproximadamente al 57.22% y 42.78% del dataset respectivamente. Aunque existe una mayor cantidad de observaciones de Osmancik, se consideró que ambas clases contaban con una representación suficiente dentro del conjunto.

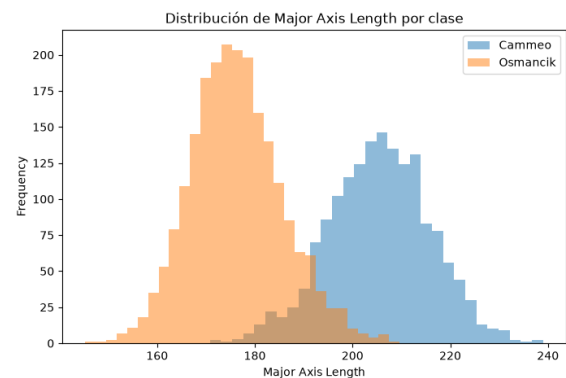
```
Clases existentes:
<StringArray>
['Cammeo', 'Osmancik']
Length: 2, dtype: str

Distribución de clases:
Class
Osmancik    2180
Cammeo      1630
Name: count, dtype: int64

Distribución porcentual de clases:
Class
Osmancik    57.217848
Cammeo      42.782152
Name: proportion, dtype: float64
```

D. Análisis de las características físicas

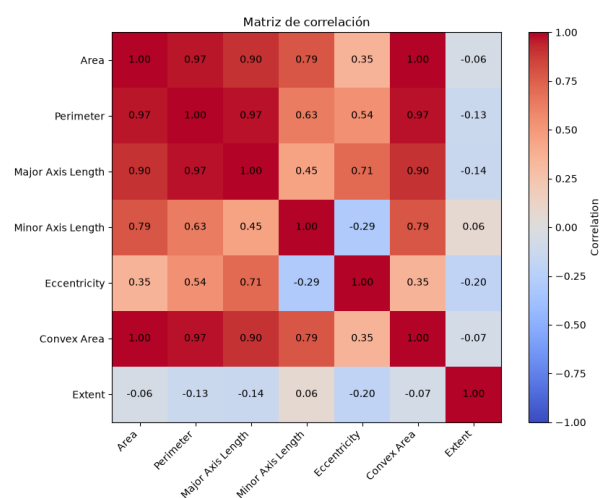
Para determinar si las características físicas presentaban diferencias entre las dos clases, se calcularon medidas descriptivas para cada variedad y se compararon sus distribuciones. Los resultados mostraron diferencias suficientemente visibles en variables relacionadas con el tamaño del grano, por ejemplo, el área promedio siendo de aproximadamente 14163 para Cammeo y 11550 para Osmancik. De igual forma, Cammeo presentó mayores valores promedio de perímetro, longitud del eje mayor y área convexa. También se observaron diferencias en características relacionadas con la forma. La excentricidad promedio fue mayor en Cammeo, mientras que Extent mostró un promedio ligeramente mayor para Osmancik. Estas diferencias se pueden interpretar como que las variables sí contienen información útil para distinguir ambas variedades.



En esta figura se puede observar esta diferencia para la variable Major Axis Length. Aunque hay una región donde las dos clases se superponen, la distribución de Cammeo se encuentra generalmente hacia valores mayores, mientras que la de Osmancik hacia valores menores. El hecho de que haya superposición también refuerza la interpretación de que una sola característica puede no ser suficiente para clasificar correctamente todas las observaciones, y justifica el uso de múltiples variables para la regresión.

E. Relaciones entre las variables

Finalmente, se calculó la matriz de correlación entre las siete características para poder identificar relaciones entre ellas. Se encontraron correlaciones positivas altas entre varias variables relacionadas con el tamaño del grano. Area y Convex Area presentaron una correlación de 0.999, mientras que Perimeter y Major Axis Length presentaron una correlación de 0.972. Asimismo, Area y Perimeter mostraron una correlación de 0.966. Sin embargo, Extent presentó correlaciones relativamente bajas con las demás características. Esto puede explicarse porque a diferencia de otras variables que miden directamente dimensiones o tamaño del grano, Extent representa la proporción entre el área ocupada por el grano y el área de su bounding box. Aunque esta característica describía un aspecto diferente de las dimensiones del grano, podía aportar información complementaria al modelo y no se descartó.



III. PREPARACIÓN DE DATOS E IMPLEMENTACIÓN DEL MODELO

A. Preparación de datos

Para aplicar el modelo de regresión logística fue necesario transformar la variable objetivo Class, debido a que sus valores originales eran etiquetas de texto "Cammeo" y "Osmancik". Es por eso que se optó por asignar el valor 0 a Osmancik y el valor 1 a Cammeo, obteniendo así una variable objetivo binaria lista para la función sigmoide usada por el modelo.

De igual forma, el dataset original se encontraba ordenado de acuerdo con la variedad de arroz, es decir, primero las observaciones de Cammeo y después las de Osmancik. Es por eso que antes de realizar la división de los datos, se les hizo un shuffle a las observaciones usando una seed para que el orden fuera igual en otras ejecuciones. Después, el 80% de las observaciones fue destinado al conjunto de train y el otro 20% al conjunto de test.

Por último, y debido a las diferencias de magnitud entre las características, se aplicó mean scaling antes del entrenamiento. Variables como Area y Convex Area contenían valores en miles de unidades, mientras que Eccentricity y Extent eran valores menores a uno. Estas diferencias provocaban problemas con la función sigmoide y el gradiente descendente, así que para evitar esos comportamientos, el escalamiento se calculó con los valores del conjunto de entrenamiento, y la misma transformación obtenida a partir de estos datos fue aplicada después al conjunto de prueba.

B. Modelo de regresión logística

Para realizar la clasificación se implementó un modelo de regresión logística usando únicamente la librería numpy para las operaciones matemáticas. El modelo calcula primero una combinación lineal de las siete características de entrada mediante un vector de pesos w y un bias b .

```
207 # MODELO DE REGRESIÓN LOGÍSTICA
208 # Función sigmoide (convierte valores a una probabilidad entre 0 y 1)
209 def sigmoid(z):
210     return 1 / (1 + np.exp(-z))
211
212 # Función para calcular la combinación lineal de las features y aplicar la sigmoide
213 def predict_probability(X, weights, bias):
214     z = np.dot(X, weights) + bias
215     return sigmoid(z)
```

Debido a que esta combinación puede producir cualquier valor real, su resultado se transforma mediante la función sigmoide, la cual convierte valores a un rango entre 0 y 1 y se interpreta como la probabilidad de que una observación pertenezca a la clase positiva. Debido a la codificación de clases que se usó, esa salida representa la probabilidad de que un grano pertenezca a la variedad Cammeo.

C. Función de costo y entrenamiento

En un inicio se le asignan pesos en ceros al modelo, al igual que el bias, y es gracias a la gradiente descendente y a cross entropy que estos cambian. Como el modelo empieza en blanco, su primera predicción es mala e "ignorante", por lo que se realiza una medición del error producido entre su estimación con la realidad con la función de cross entropy:

$$H(p, q) = - \sum_i p_i \log q_i = - y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

En esta función, m representa el número de observaciones, y_i la clase real y $\hat{y}_i$ la probabilidad estimada por el modelo. En el código, la implementación se realizó con operaciones de numpy. De igual forma, para evitar valores indefinidos producidos por $\log(0)$, las probabilidades se limitaron a un intervalo ligeramente mayor a cero e inferior a uno antes de calcular la función de costo.

```
219 # Función para calcular el costo mediante cross entropy
220 def binary_cross_entropy(y, y_hat):
221     epsilon = 1e-15
222
223     # Se evitan valores iguales a 0 o 1 para no calcular log(0)
224     y_hat = np.clip(y_hat, epsilon, 1 - epsilon)
225
226     return -np.mean(y * np.log(y_hat) + (1 - y) * np.log(1 - y_hat))
```

En cada epoch se calcularon las probabilidades de todas las observaciones de entrenamiento y después la diferencia entre las predicciones y sus valores reales. A partir de este error se calcularon los gradientes correspondientes a los pesos y al bias.

```
247 # 3. Gradientes
248 error = y_hat - y
249
250 dw = (1 / n_samples) * np.dot(X.T, error)
251 db = (1 / n_samples) * np.sum(error)
```

En el código, el vector dw representa el gradiente del costo respecto a los pesos, y db representa el gradiente correspondiente al bias.

Finalmente, los parámetros se actualizaron en dirección contraria al gradiente con el objetivo de reducir progresivamente la función de costo.

```
253 # 4. Actualización de parámetros
254 weights = weights - learning_rate * dw
255 bias = bias - learning_rate * db
```

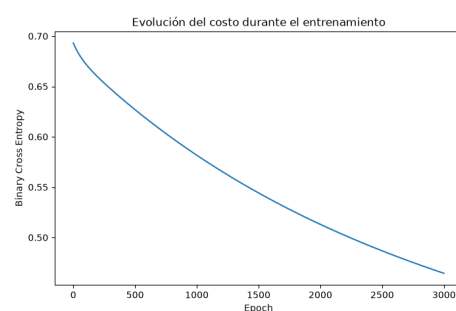
D. Configuración del entrenamiento

Para el caso base presentado en este avance se utilizó una tasa de aprendizaje de 0.03 y se realizaron 3000 epochs de entrenamiento. Durante cada epoch se almacenó el valor de la cross entropy para así poder analizar la evolución del proceso de optimización. Al finalizar el entrenamiento se utilizaron los parámetros obtenidos para hacer predicciones tanto sobre el conjunto de entrenamiento como sobre el conjunto de prueba.

IV. RESULTADOS INICIALES

A. Convergencia del modelo

Durante el entrenamiento se registró el valor de la función de costo en cada epoch para así poder observar el comportamiento del gradiente descendente. El modelo comenzó con una cross entropy de aproximadamente 0.693 y presentó una disminución a lo largo de las 3000 epochs. Alrededor de la epoch 2900, el costo había disminuido hasta 0.468.



Epoch 2900, Loss: 0.468562

La función de costo mantiene una tendencia a decrecer y no presenta oscilaciones, lo cual indica que la tasa de aprendizaje utilizada permitió que el gradiente descendente actualizara los parámetros correctamente en una dirección que reduce el error del modelo. Sin embargo, la curva todavía presenta una pendiente negativa al finalizar las 3000 epochs, por lo que la optimización podría continuar reduciendo el costo si hubiera un mayor número de iteraciones.

B. Resultados de la clasificación

Una vez finalizado el entrenamiento, el modelo obtuvo una accuracy de 88.16% con el conjunto de entrenamiento y de 87.66% con el de prueba. Esto significa que aproximadamente 88 de cada 100 observaciones no usadas para ajustar los parámetros fueron clasificadas correctamente como Cammeo u Osmancik.

La diferencia entre la accuracy obtenida en entrenamiento y prueba fue de aproximadamente 0.49%, y esto se puede interpretar como que, para esta división de los datos, el desempeño del modelo se mantiene al utilizar observaciones que no participaron durante el entrenamiento. Así mismo, se presentan algunos ejemplos de las predicciones realizadas sobre el conjunto de prueba.

```
Accuracy train: 88.15616797900262
Accuracy test: 87.66404199475065
Ejemplo 1: Probabilidad Cammeo = 0.2714, Predicción = Osmancik, Real = Osmancik
Ejemplo 2: Probabilidad Cammeo = 0.4533, Predicción = Osmancik, Real = Cammeo
Ejemplo 3: Probabilidad Cammeo = 0.2275, Predicción = Osmancik, Real = Osmancik
Ejemplo 4: Probabilidad Cammeo = 0.1928, Predicción = Osmancik, Real = Osmancik
Ejemplo 5: Probabilidad Cammeo = 0.1981, Predicción = Osmancik, Real = Osmancik
Ejemplo 6: Probabilidad Cammeo = 0.7854, Predicción = Cammeo, Real = Cammeo
Ejemplo 7: Probabilidad Cammeo = 0.2602, Predicción = Osmancik, Real = Osmancik
Ejemplo 8: Probabilidad Cammeo = 0.7782, Predicción = Cammeo, Real = Cammeo
Ejemplo 9: Probabilidad Cammeo = 0.3726, Predicción = Osmancik, Real = Osmancik
Ejemplo 10: Probabilidad Cammeo = 0.1669, Predicción = Osmancik, Real = Osmancik
```